



In [1]:

```
!pip -q install qiskit qiskit-aer
```

9.8/9.8 MB	32.0 MB/s	eta 0:00:00
12.4/12.4 MB	32.7 MB/s	eta 0:00:00
2.4/2.4 MB	21.1 MB/s	eta 0:00:00
54.9/54.9 kB	1.8 MB/s	eta 0:00:00

In [2]:

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector, partial_trace
from collections import Counter
import pandas as pd
import numpy as np
```

Easy: Construct the Φ^+ Bell state and confirm through 1024-shot simulation that only '00' and '11' outcomes occur.

In [3]:

```
# Create a 2-qubit circuit
qc = QuantumCircuit(2, 2)

# Create  $\Phi^+ = (|00\rangle + |11\rangle) / \sqrt{2}$ 
qc.h(0)
qc.cx(0, 1)

# Measure both qubits
qc.measure([0, 1], [0, 1])

print(qc)
simulator = AerSimulator()

compiled_circuit = transpile(qc, simulator)

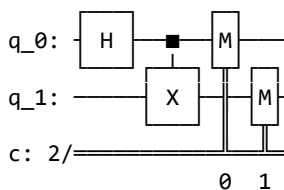
result = simulator.run(
    compiled_circuit,
    shots=1024
).result()

counts = result.get_counts()

print("Measurement results:")
print(counts)
allowed = {'00', '11'}

unexpected = set(counts.keys()) - allowed

if len(unexpected) == 0:
    print("Confirmed: only 00 and 11 occurred.")
else:
    print("Unexpected outcomes:", unexpected)
```



Measurement results:
{'00': 511, '11': 513}
Confirmed: only 00 and 11 occurred.

Medium: Construct all four Bell states and tabulate the

measurement outcome distributions for comparison.

In [7]:

```
def bell_circuit(state):
    """
    state:
        'Phi+' -> ( $|00\rangle + |11\rangle$ ) /  $\sqrt{2}$ 
        'Phi-' -> ( $|00\rangle - |11\rangle$ ) /  $\sqrt{2}$ 
        'Psi+' -> ( $|01\rangle + |10\rangle$ ) /  $\sqrt{2}$ 
        'Psi-' -> ( $|01\rangle - |10\rangle$ ) /  $\sqrt{2}$ 
    """

    qc = QuantumCircuit(2, 2)

    # Create Phi+
    qc.h(0)
    qc.cx(0, 1)

    if state == 'Phi-':
        qc.z(0)

    elif state == 'Psi+':
        qc.x(1)

    elif state == 'Psi-':
        qc.x(1)
        qc.z(0)

    qc.measure([0, 1], [0, 1])

    return qc

# Define the bell states
bell_states = ['Phi+', 'Phi-', 'Psi+', 'Psi-']

# Initialize a simulator
simulator = AerSimulator()

results = {}

# Simulate each Bell state
for state in bell_states:
    qc = bell_circuit(state)
    compiled_circuit = transpile(qc, simulator)
    job = simulator.run(compiled_circuit, shots=1024)
    result = job.result()
    counts = result.get_counts(compiled_circuit)
    results[state] = counts

all_outcomes = ['00', '01', '10', '11']

table = []
for state in bell_states:
    counts = results[state]

    row = {
        'Bell State': state,
        '00': counts.get('00', 0),
        '01': counts.get('01', 0),
        '10': counts.get('10', 0),
        '11': counts.get('11', 0)
    }

    table.append(row)

df = pd.DataFrame(table)

df
```

	Bell State	00	01	10	11
--	------------	----	----	----	----

	Bell State	00	01	10	11
0	Phi+	502	0	0	522
1	Phi-	488	0	0	536
2	Psi+	0	531	493	0
3	Psi-	0	511	513	0

Hard: Measure a Bell pair in the Hadamard basis instead of the computational basis and explain the resulting correlation pattern.

In [8]:

```
def bell_hadamard_measurement(state):
    qc = QuantumCircuit(2, 2)

    # Prepare Bell state
    qc.h(0)
    qc.cx(0, 1)

    if state == 'Phi-':
        qc.z(0)

    elif state == 'Psi+':
        qc.x(1)

    elif state == 'Psi-':
        qc.x(1)
        qc.z(0)

    # Change from Hadamard basis to computational basis
    qc.h(0)
    qc.h(1)

    # Measure
    qc.measure([0, 1], [0, 1])

    return qc
hadamard_results = {}

for state in bell_states:

    qc = bell_hadamard_measurement(state)

    compiled = transpile(qc, simulator)

    result = simulator.run(
        compiled,
        shots=1024
    ).result()

    hadamard_results[state] = result.get_counts()

hadamard_results
table = []

for state in bell_states:
    counts = hadamard_results[state]

    table.append({
        'Bell State': state,
        '00': counts.get('00', 0),
        '01': counts.get('01', 0),
        '10': counts.get('10', 0),
        '11': counts.get('11', 0)
    })

hadamard_df = pd.DataFrame(table)
```

hadamard_df

	Bell State	00	01	10	11
0	Phi+	528	0	0	496
1	Phi-	0	536	488	0
2	Psi+	522	0	0	502
3	Psi-	0	524	500	0

Real-world: Relate Bell-state correlations conceptually to Quantum Key Distribution (QKD) and prepare a short explanation of how entanglement could secure a communication channel.

In [9]:

```
qc = QuantumCircuit(2, 2)

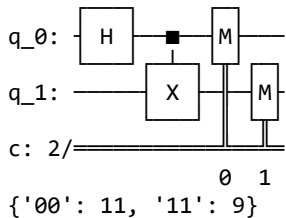
# Prepare Phi+
qc.h(0)
qc.cx(0, 1)

# Alice and Bob measure in computational basis
qc.measure([0, 1], [0, 1])

print(qc)

result = simulator.run(
    transpile(qc, simulator),
    shots=20
).result()

print(result.get_counts())
```



Challenge: Implement a 3-qubit GHZ and a 3-qubit W-state, compare their entanglement structure by examining behaviour when one qubit is measured/traced out.

In [10]:

```
ghz = QuantumCircuit(3, 3)

ghz.h(0)
ghz.cx(0, 1)
ghz.cx(0, 2)

ghz.measure([0, 1, 2], [0, 1, 2])

print(ghz)
result = simulator.run(
    transpile(ghz, simulator),
    shots=1024
).result()

ghz_counts = result.get_counts()
```

```

print(ghz_counts)
from qiskit.circuit.library import StatePreparation

w_vector = np.zeros(8, dtype=complex)

# Qiskit's basis ordering:
#  $|001\rangle$ ,  $|010\rangle$ ,  $|100\rangle$ 
w_vector[1] = 1 / np.sqrt(3)
w_vector[2] = 1 / np.sqrt(3)
w_vector[4] = 1 / np.sqrt(3)

w_circuit = QuantumCircuit(3, 3)

w_circuit.append(
    StatePreparation(w_vector),
    [0, 1, 2]
)

w_circuit.measure([0, 1, 2], [0, 1, 2])

print(w_circuit)
result = simulator.run(
    transpile(w_circuit, simulator),
    shots=1024
).result()

w_counts = result.get_counts()

print(w_counts)
comparison = pd.DataFrame({
    'State': ['GHZ', 'W'],
    'Expected outcomes': [
        '000, 111',
        '001, 010, 100'
    ],
    'Number of components': [
        2,
        3
    ]
})

comparison
ghz_no_measure = QuantumCircuit(3)

ghz_no_measure.h(0)
ghz_no_measure.cx(0, 1)
ghz_no_measure.cx(0, 2)

ghz_state = Statevector.from_instruction(ghz_no_measure)

# Trace out qubit 2
ghz_reduced = partial_trace(ghz_state, [2])

print("GHZ reduced density matrix:")
print(ghz_reduced)
w_no_measure = QuantumCircuit(3)

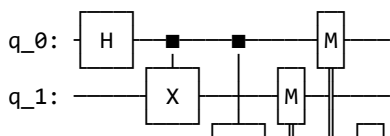
w_no_measure.append(
    StatePreparation(w_vector),
    [0, 1, 2]
)

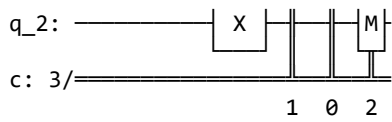
w_state = Statevector.from_instruction(w_no_measure)

# Trace out qubit 2
w_reduced = partial_trace(w_state, [2])

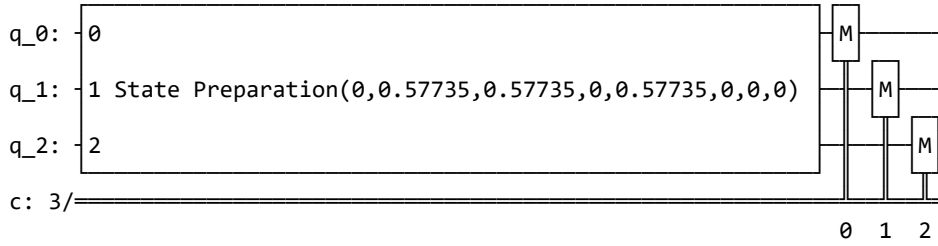
print("W reduced density matrix:")
print(w_reduced)

```





{'000': 516, '111': 508}



{'010': 352, '100': 321, '001': 351}

GHZ reduced density matrix:

```
DensityMatrix([[0.5+0.j, 0. +0.j, 0. +0.j, 0. +0.j],
               [0. +0.j, 0. +0.j, 0. +0.j, 0. +0.j],
               [0. +0.j, 0. +0.j, 0. +0.j, 0. +0.j],
               [0. +0.j, 0. +0.j, 0. +0.j, 0.5+0.j]],
               dims=(2, 2))
```

W reduced density matrix:

```
DensityMatrix([[ 3.33333333e-01+0.00000000e+00j,
                 -1.24708365e-16+3.20493781e-17j,
                 -1.23076476e-17+1.67411225e-16j,
                 2.56116886e-32-3.61783048e-33j],
               [-1.24708365e-16-3.20493781e-17j,
                 3.33333333e-01+0.00000000e+00j,
                 3.33333333e-01-3.20493781e-17j,
                 -5.07749235e-17-1.60246891e-17j],
               [-1.23076476e-17-1.67411225e-16j,
                 3.33333333e-01+3.20493781e-17j,
                 3.33333333e-01+0.00000000e+00j,
                 -5.07749235e-17-1.60246891e-17j],
               [ 2.56116886e-32+3.61783048e-33j,
                 -5.07749235e-17+1.60246891e-17j,
                 -5.07749235e-17+1.60246891e-17j,
                 8.50465054e-33+0.00000000e+00j]],
               dims=(2, 2))
```